

深度优先搜索

深度优先搜索，顾名思义，就是按照深度优先来搜索，就是一条路走到黑，直到这条路走不通了或者走到一个以前走到的位置才返回上一层，然后尝试其他路径。

```
dfs(int x,int y)//当前位置
{
    int next[4][2]={{0,1},{1,0},{0,-1},{-1,0}};
    if(x==X&&y==Y)//如果到达终点
    ...
    for(int k=0;k<4;k++)//枚举四个方向
    {
        int tx=x+next[k][0];
        int ty=y+next[k][1];
        if(tx<1||tx>n||ty<1||ty>m)//判断是否越界
        continue;
        if(!book[x][y]&&map[x][y])//判断是否可行
        {
            book[tx][ty]=1;//标记
            dfs(tx,ty);
            book[tx][ty]=0;//回溯，取消标记
        }
    }
}
```

深度优先搜索

深度优先搜索的原理就是按照给定的规律去尝试所有的可能性。如果可以行就继续尝试下去，如果不行就返回上一步换一个方向尝试。

根据深搜的这种性质，我们可以用来暴力求解很多排列组合问题，虽然不一定是AC算法，但是一般可以得部分。比如八皇后，放苹果，拆分数... 他都是尝试所有的可能然后得出正确的答案。

但是在实际的比赛中，很多时候都说不是求解所有方案，而且要求得到最优解，即从所有的方案里面找出最优的一种，这个时候虽然裸的DFS是可以的，但是很费时间，有的方案明显不是最优的，我们就可以直接在中途就舍弃，这种方法被称之为**剪枝**。



剪 枝

剪枝一定要遵循的三个原则：

- 1、正确性
 - 2、准确性
 - 3、高效性
- 

剪枝

剪枝，就是减小搜索树的范围，尽早排除搜索树中不必要的部分，达到更快得到结果的过程。以最优解(最短路)为例，剪枝的方法有以下几种：

一：可行性剪枝

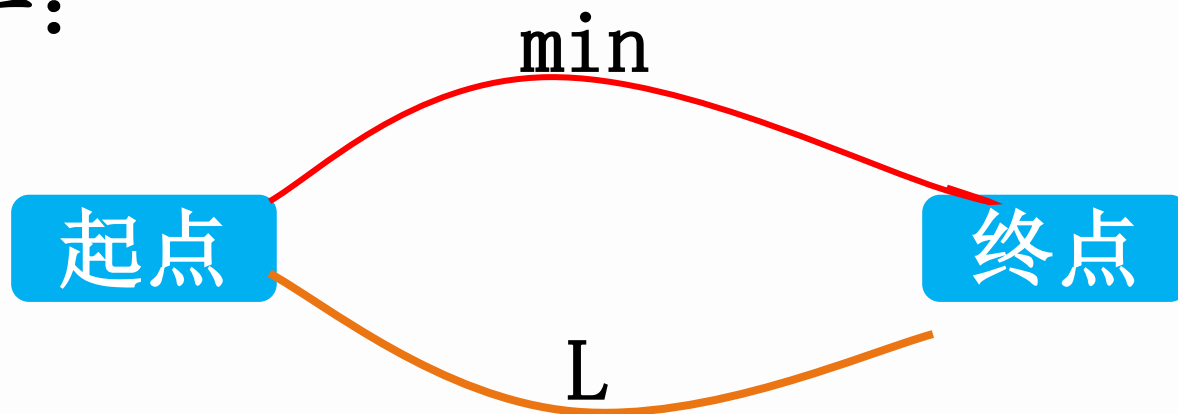
判断下一个位置是否可以走。比如位置越界，下一个位置是障碍物，或者下一个位置已经走过(死循环)。

二：最优性剪枝

因为我们是求解最优解，所以如果在中途就发现当方案不可能是最优的(比前面最优方案差)，直接舍弃当前方案，执行回溯。这是搜索题最基本的剪枝方法。

最优性剪枝

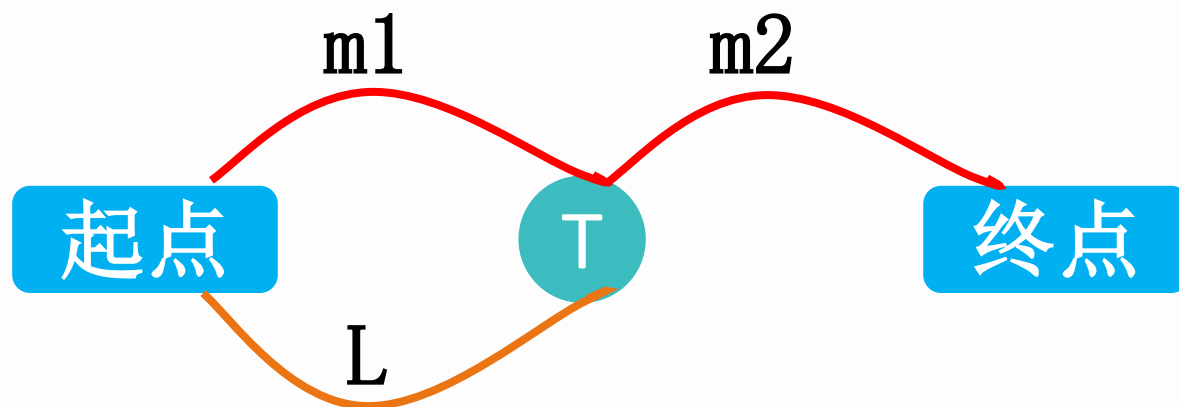
方法一：



当前求出的最优解为min，尝试其他方案的时候，中途路径长度 $step \geq min$. 直接回溯

```
if(step>=min)
return ;
if(x==X&&y==Y)
min=step;
```

最优性剪枝



在已知最优解中，起点到某一点的最短路径为 $m1$ ，如果其他路径中起点到该点的距离 $L > m1$ ，也不需要再往下搜索了。

```
if(step >= dis[x][y])  
    return ;  
else  
    dis[x][y] = step;
```

dis[x][y]: 起点到点(x, y)的当前最短距离。搜到一个点，要么更新，要么回溯。

剪枝

这两种剪枝方法都是NOIP必须掌握的，第二种方法明显远优于第一种，所以在考试的时候能用第二种剪枝方法就用第二种。但是第一种剪枝方法适用面广一些，有些情况第二种剪枝方法是无法使用的，比如不固定起点的问题。除此之外，还有其他的剪枝方法，这些剪枝方法都是很重要的，决定着一道题的暴力分的高低。

三：排除等效冗余

如果有几种看似不同的选择效果是完全等价的，我们就可以只选择一种进行尝试。比如P2386 放苹果。我们会发现第一个盘子放a个，第二个盘子放b个，和反过来放效果是等价的，我们就不需要考虑了。可以节省很多时间

$$A(n, m) = C(n, m) * A(m, m)$$

剪枝

四：记忆化

记录每个状态的搜索结果，在重复遍历一个状态时直接检索并返回，比如**P1028 数的计算**，就需要记录每个数可以分解的数的个数，当下次分解到该数的时候就直接检索。有的时候还需要记录多个点的状态，这个时候可以考虑状态压缩，以免使用多维数组超出内存限制。

五：优化搜索顺序

在一些搜索问题中，搜索的顺序不一样，搜索的时间也会不同，我们应该分析问题，选择合理的搜索顺序，而不是按部就班的从1搜索到n。比如**POJ 3074 数独游戏**。玩过数独的同学都知道，填数独肯定不是依次填，而是先选择可能性最少的先填，这些效率会高很多。

小猫爬山

Freda和rainbow饲养了 N 只小猫，这天，小猫们要去爬山。经历了千辛万苦，小猫们终于爬上了山顶，但是疲倦的它们再也不想徒步走下山了（呜咕>_<）。

Freda和rainbow只好花钱让它们坐索道下山。索道上的缆车最大承重量为 W ，而 N 只小猫的重量分别是 C_1 、 C_2 …… C_N 。当然，每辆缆车上的小猫的重量之和不能超过 W 。每租用一辆缆车，Freda和rainbow就要付1美元，所以他们想知道，最少需要付多少美元才能把这 N 只小猫都运送下山？

Codevs 4228: 小猫爬山

分割矩阵

题目描述

展开

将一个 $a*b$ 的数字矩阵进行如下分割：将原矩阵沿某一条直线分割成两个矩阵，再将生成的两个矩阵继续如此分割（当然也可以只分割其中的一个），这样分割了 $(n-1)$ 次后，原矩阵被分割成了 n 个矩阵。（每次分割都只能沿着数字间的缝隙进行）

原矩阵中每一位置上有一个分值，一个矩阵的总分为其所含各位置上分值之和。现在需要把矩阵按上述规则分割成 n 个矩阵，并使各矩阵总分的均方差最小。

请编程对给出的矩阵及 n ，求出均方差的最小值。

输入格式

第一行为3个整数，表示 $a,b,n(1 < a,b \leq 10, 1 < n \leq 10)$ 的值。

第二行至第 $n+1$ 行每行为 b 个小于100的非负整数，表示矩阵中相应位置上的分值。每行相邻两数之间用一个空格分开。

输出格式

仅一个数，为均方差的最小值（四舍五入精确到小数点后2位）

分割矩阵

该题很容易猜到是一道动态规划的题，但是如果我们的动态规划的水平并不高，写不出来状态转移方程，应该怎么办呢？

那很明显就是写爆搜。

对于当前的一个矩形 (a, b, c, d) ，分割的时候可以横着分割，割线范围 $[a, c)$ （分成 (a, b, i, d) 和 $(i+1, b, c, d)$ ），也可以竖着分割，割线范围 $[b, d)$ （分成 (a, b, c, i) 和 $(a, i+1, c, d)$ ），那么爆搜就写出来了。

对于当前的一个矩形 (a, b, c, d) ，需要隔成 k 个矩形，可以先分一次，分成两个子矩形，然后在让两个子矩形分割成的矩形数量之和 $= k$ 。

分割矩阵

```
for(int i=a;i<c;i++)//如果横着切
{
    for(int j=1;j<num;j++)//其中一个矩形分割成j块, 另一个num-j块
    {
        double tmp=dfs(a,b,i,d,j)+dfs(i+1,b,c,d,num-j);
        dp[a][b][c][d][num]=min(dp[a][b][c][d][num],tmp);
    }
}
for(int i=b;i<d;i++)//如果竖着切
{
    for(int j=1;j<num;j++)
    {
        double tmp=dfs(a,b,c,i,j)+dfs(a,i+1,c,d,num-j);
        dp[a][b][c][d][num]=min(dp[a][b][c][d][num],tmp);
    }
}
```

分割矩阵

既然是搜索那么肯定要剪枝，如果同样的一块矩形 (a, b, c, d) 被分割成 k 块，那么肯定取搜索到的最小的答案，所以我们可以记录一个 $dp[a][b][c][d][k]$ 表示左上角顶点为 (a, b) ，右下角顶点为 (c, d) 的矩形被分割成 k 块的最小值。所以我们可以记忆化。

```
if(dp[a][b][c][d][num])
return dp[a][b][c][d][num];
if(num==1)//分割成一个矩阵的值可以直接算出来
{
    int tmp=js(a,b,c,d);
    return (tmp-ave)*(tmp-ave);
}
```

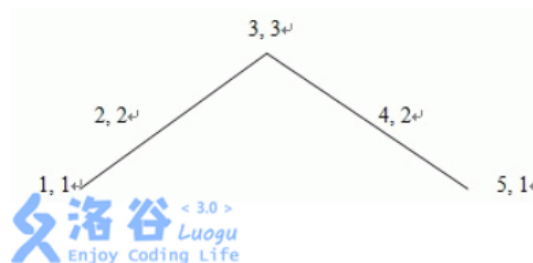
天天酷跑

题目描述

展开

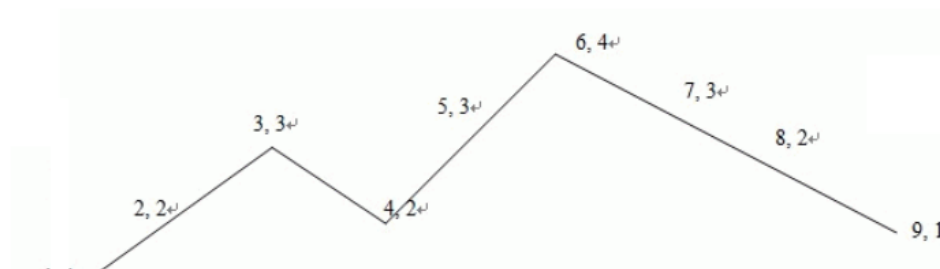
在游戏天天酷跑中，最爽的应该是超级奖励模式了吧，没有一切障碍，可以尽情的吃金币，现在请你控制游戏角色来获得尽可能多的分数。游戏界面离散为一个长度为 $1 \sim n$ ，高度为 $1 \sim m$ （初始点为 $(0, 1)$ ）的矩阵图。每个格子上都有收益 $(-1 \sim 1000)$ ， -1 表示该点不能通过。游戏角色从起点一路奔跑向终点，中途可以跳跃来获得更高的分数，在空中还能进行连跳。

游戏开始前你可以设定跳跃的高度，以及能连跳的次数，初始跳跃高度为1，连跳数为1（最多为5），升级跳跃高度和连跳都需要一定的花费。跳跃高度设定完后游戏角色每次跳跃高度都将固定，连跳必须在下落过程中可以使用。所有操作都将在整点上完成，需要保证设定完的跳跃高度及连跳数，无法跳出游戏高度上限。



从 $(1, 1)$ 点用一次跳跃，一次经过 $(2, 2), (3, 3), (4, 2), (5, 1)$ 。

以下是连跳数为2连跳，跳跃高度为2的跳跃方案：



天天酷跑

该题和上一道题实际上是一样的，因为每次跳跃的高度和连续跳跃的次数并没有单调性，所以直接枚举就可以了，对于枚举的每一种情况，去搜索。

对于当前位置 (x, y) ，如果最多还可以跳跃 k 次，如果前面已经得到了这个结果，那么就可以直接使用。所以我们记录一下 $dp[x][y][k]$ 。

对于当前这个状态，后续有三种可能，如果还可以跳跃，那么可以跳，如果不可以跳跃，如果已经在地面上了，那么久平移，如果在空中，就往下掉，最多就这三种情况，然后对答案取一个最大值，并把它赋值给 $dp[x][y][k]$ 就可以了。

注意边界，以及行列和坐标的区别。

道路

题目描述

展开

W 国的交通呈一棵树的形状。W 国一共有 $n - 1$ 个城市和 n 个乡村，其中城市从 1 到 $n - 1$ 编号，乡村从 1 到 n 编号，且 1 号城市是首都。道路都是单向的，本题中我们只考虑从乡村通往首都的道路网络。对于每一个城市，恰有一条公路和一条铁路通向这座城市。对于城市 i ，通向该城市的道路（公路或铁路）的起点，要么是一个乡村，要么是一个编号比 i 大的城市。没有道路通向任何乡村。除了首都以外，从任何城市或乡村出发只有一条道路；首都没有往外的道路。从任何乡村出发，沿着唯一往外的道路走，总可以到达首都。

W 国的国王小 W 获得了一笔资金，他决定用这笔资金来改善交通。由于资金有限，小 W 只能翻修 $n - 1$ 条道路。小 W 决定对每个城市翻修恰好一条通向它的道路，即从公路和铁路中选择一条并进行翻修。小 W 希望从乡村通向城市可以尽可能地便利，于是根据人口普查的数据，小 W 对每个乡村制定了三个参数，编号为 i 的乡村的三个参数是 a_i ， b_i 和 c_i 。假设从编号为 i 的乡村走到首都一共需要经过 x 条未翻修的公路与 y 条未翻修的铁路，那么该乡村的不便利值为

$$c_i \cdot (a_i + x) \cdot (b_i + y)$$

在给定的翻修方案下，每个乡村的不便利值相加的和为该翻修方案的不便利值。翻修 $n - 1$ 条道路有很多方案，其中不便利值最小的方案称为最优翻修方案，小 W 自然希望找到最优翻修方案，请你帮助他求出这个最优翻修方案的不便利值。

输入格式

第一行为正整数 n 。

接下来 $n - 1$ 行，每行描述一个城市。其中第 i 行包含两个数 s_i, t_i 。 s_i 表示通向第 i 座城市的公路的起点， t_i 表示通向第 i 座城市的铁路的起点。如果 $s_i > 0$ ，那么存在一条从第 s_i 座城市通往第 i 座城市的公路，否则存在一条从第 $-s_i$ 个乡村通往第 i 座城市的公路； t_i 类似地，如果 $t_i > 0$ ，那么存在一条从第 t_i 座城市通往第 i 座城市的铁路，否则存在一条从第 $-t_i$ 个乡村通往第 i 座城市的铁路。

接下来 n 行，每行描述一个乡村。其中第 i 行包含三个数 a_i, b_i, c_i ，其意义如题面所示。

道路

这是一道树上求解的题，树上贪心明显过不了，那么很容易想到是树形dp，如果我们直接自下而上的树形dp那么明显是有问题的，下面的结果存储了之后*路径条数我们根本不知道是多少，所以只能自上而下的树形dp。

如果dp学得好点的同学可能还比较好做，但是如果dp学得差得估计就难受了。

我们还是可以考虑记忆化搜索，思路肯定都是从根结点开始往下面搜，要么砍连接左儿子的公路，要么砍连接右儿子的铁路。对于当前点id, 到起点有公路x条，铁路y条，那么如果接下里砍公路，那么左儿子就是(x, y), 右儿子就是(x, y+1), 如果接下来砍铁路，那么左儿子就是(x+1, y), 右儿子就是(x, y)。

道路

接下来就考虑记忆化搜索了，对于一个点，对他有影响的只有到起点的公路和铁路分别的总条数，和哪一条是公路，哪一条是铁路没有关系，比如公路—铁路和铁路—公路，对答案影响一样的，所以我们只需要开一个 $dp[id][x][y]$ 表示 id 这个点到起点有 x 条公路， y 条铁路的最小代价就可以了。

剩下的就是记忆化搜索的裸题了。注意省空间。

道路

```
long long dfs(int id,int x,int y)//从起点到达id这个点经过了x条公路, y条铁路
{
    if(id>=n)//乡村是叶子结点不用往下找了
        return c[id]*(x+a[id])*(y+b[id]);
    if(dp[id][x][y]!=inf)
        return dp[id][x][y];
    long long t1=dfs(son[id][0],x,y)+dfs(son[id][1],x,y+1);//砍公路
    long long t2=dfs(son[id][0],x+1,y)+dfs(son[id][1],x,y);//如果砍铁路
    return dp[id][x][y]=min(t1,t2);
}
```

玄学搜索

这种方法在很多时候会起到意料之外的作用。大概思路就是不要按照常规的顺序或者按照题目中给出的顺序搜索，因为测试数据不是随机出的，肯定会卡一些暴力算法，其中卡得最严的就是顺序搜索或者题目中给定的顺序搜索，很多时候反其道而行得分会比较高，至少不会低。因为测试数据就是一般就10组，出题人不可能写很多不同搜索顺序的代码一个一个去测试是否都可以卡住。毕竟大学生是最懒的人，越聪明的人潜意识里越想偷懒。

比如P1074 靶形数独 从 $(1, 1)$ 搜索到 (n, n) 得分70，直接反过来从 (n, n) 搜索到 $(1, 1)$ 得分95，再卡一下次数，剪一下枝就可以AC了。

折半搜索

深度优先搜索的效率是特别低的，比如 n 个物品，选或者不选的最优解，那么最坏时间复杂度是 2^n ，就算加上剪枝也很慢。所以这种情况最多过25左右。但是如果 n 在50以内呢？这里有一种比较优秀的方法就是折半搜索。

比如当 $n=40$ 的时候，我们先搜索前半，把所有可能的情况都存储起来，那么最多 $2^{20}=100w$ 种可能，然后再对答案去重排序。然后我们再去搜索后半，对于后面的每一种情况，我们去前面的答案中二分查找一个我们需要的最优解，前后两部分加起来的答案就是我们当前的最优解，那么时间复杂度为 $2^n \log(2^n)$ ，再结合前面讲的剪枝的方法，对于50以内的数据可以跑过。

JOY OI 送礼物

作为惩罚，GY被遣送去帮助某神牛给女生送礼物(GY：貌似是个好差事)但是在GY看到礼物之后，他就不这么认为了。某神牛有N个礼物，且异常沉重，但是GY的力气也异常的大(-_-b)，他一次可以搬动重量和在 w ($w \leq 2^{31}-1$) 以下的任意多个物品。GY希望一次搬掉尽量重的一些物品，请你告诉他在他的力气范围内一次性能搬动的最大重量是多少。

第一行两个整数，分别代表W和N。 ($N \leq 45$)

以后N行，每行一个正整数表示 $G[i]$, $G[i] \leq 2^{31}-1$ 。

仅一个整数，表示GY在他的力气范围内一次性能搬动的最大重量。

送礼物

拿到题一看， n 件物品，拿取其中 k 件质量之和不超 w ，很明显是背包，再看一下数据范围， $2^{31}-1$ ，肯定没法做。如果用搜索呢？ $n \leq 45$ 。一共右 2^{45} 种情况，也超了，但是加上各种优化剪枝说不定可以。但是实际上仍然会超时，我们可以考虑一下 2^{45} 虽然会超，但是 2^{23} 是可以的，所以就可以考虑双向搜索。先dfs前半段，存储所有的可能结果，然后再dfs后判断，对于后面的每一种情况，在前面的情况中快速找到之和 $< w$ 的那组数据，并从中求得最大值。查找很明显可以先排序再用二分查找，时间复杂度为 $2^{23} * \log(2^{23})$ ，会超出一点，剩下的就只能从剪枝中想办法了。

剪枝

一：要使后面选择的物品可能性更小，很明显选择物品的时候从大到小选择。

二：如果刚好选择物品质量= w 直接强行退出程序

三：为了方便后期查找，先排序

四：为了减小搜索规模，序列去重

五：二分查找前半段

剩下的，自己去想，不然仍然会被卡点

迭代加深搜索

我们知道，深搜的用途比较广，比较节省空间，但是效率比较低；广搜虽然效率比较高，但是耗费空间比较大，用途也稍微窄一点。那么有没有一种搜索能够结合深搜和广搜的优点呢？

后来人们在研究中，发明了一种新的搜索方法——迭代加深搜索。他在很多时候会有意想不到的效果。

迭代加深搜索其实很简单，就是DFS+BFS，他先假设答案就在第 k 层，那么从起点开始以深搜的方式搜索第 k 层的所有点，如果没有搜到，那么就假设答案在第 $k+1$ 层，再以深搜的方式去搜索去 $k+1$ 层的所有点。一直循环下去，直到有解为止。

迭代加深搜索又可以理解为按层拓展的深度优先搜索。

迭代加深搜索

```
int k=1;//先从第一层开始搜索
while(1)
{
    dfs(1,k);//假设答案就在第k层，那么搜到第k层就返回
    if(flag==0)//搜完第k层都没有搜到答案
        k++;
    else//答案就在第k层，跳出循环
        break;
}
```

POJ 3134

题目大意：给定一个数 x ，问如何通过乘除法变为 x^n . 最少次数是多少？每次可以乘以或除以已经被构造出来的数。

比如 $x^2 = x * x$ (1次)

$x^3 = x^2 * x$ (2次)

$x^4 = x^2 * x^2$ (2次)

$x^5 = x^2 * x^3$ (3次)

x^6 (3次)

x^7 (4次)

x^{29} (7次)

x^{30} (6次)

x^{31} (6次)

解题思路

该题因为有多组数据，直接用深搜，那么剪枝只能用 $n-1$ 来剪枝(每次都乘以 x ，最多 $n-1$ 次就可以变为 x^n)。但是数很多的情况下，随机数据很多时候根本用不到 $n-1$ 次，特别是后期 n 比较大的时候，真实次数远小于 $n-1$ 。所以直接深搜效率很多，所以我们可以考虑迭代加深搜索。

```
num[0]=1; // 记录所有出现过的次数
int k=1;
while(1)
{
    if(dfs(0,1,k))
        break;
    k++;
}
printf("%d\n",k);
```

解题思路

```
bool dfs(int cur,int x,int dep)//操作次数, 最大指数
{
    if(num[cur]==n) return 1;
    if(cur>=dep) return 0;
    x=max(x,num[cur]);
    for(int i=cur;i>=0;i--)
    {
        num[cur+1]=num[cur]+num[i];//乘法
        if(dfs(cur+1,x,dep))
            return 1;
        if(num[cur]>num[i])
            num[cur+1]=num[cur]-num[i];//除法
        else
            num[cur+1]=num[i]-num[cur];//除法
        if(dfs(cur+1,x,dep))
            return 1;
    }
}
```

解题思路

但是这样做效率还是很低，虽然比普通的深搜效果好一些，所以我们在使用迭代加深搜索的时候还是要考虑启发式搜索的思想。考虑一下当前状态到最终状态最少需要多少步？

这个很容易想到，对于前面出现的最大次数，每次平方，最少需要平方多少次。就是最快的方法。

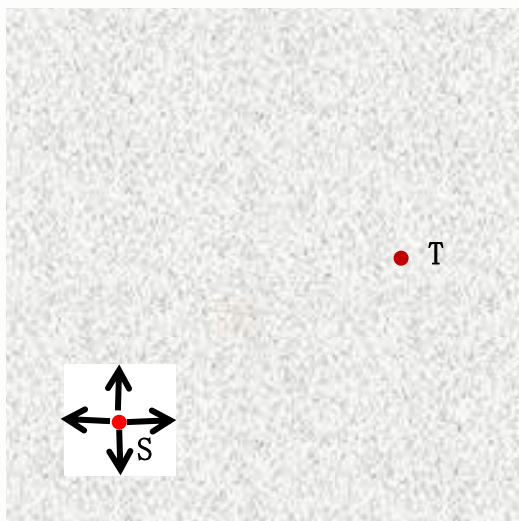
所以要加上这个剪枝。

```
if(x*(1<<(dep-cur))<n)//如果最快增长在第dep层都到不了目标  
return 0;
```

启发式搜索

我们先思考一下深搜和广搜慢的原因：对于每一个位置，有多个方向可以选择，我们一般都是给定固定的选择顺序，也就是随机性的搜索下去，这样的话，就很容易给出卡算法的数据，那能不能根据上面的搜索做一个优化呢？

很容易想到一种优化方法，并且也是我们在生活中经常用到的一种办法，只是我们没注意而已。



我们想象一下如果身处一个迷宫之中，我们知道终点在你的右上角，那么此时你面前有上下左右四条路可以选择，那么你会先选择哪一条呢？很明显，我们会主观的先选择向上或者向右的那一条。

启发式搜索

这就是普通搜索的一个简单的优化，但是实际上没啥用，因为完全可以出几组数据让往左上角最终走到死胡同，而往右下角迂回才是正解。这样效率并没有太大的变化。

前面这种搜索我们可以理解为贪心式搜索，每次只考虑当前最优，即局部最优，但是和贪心一样，局部最优并不是整体最优，所以我们还要考虑一下后面的情况。

但是后面的路径我们都还没有尝试过，肯定是没有办法知道最短路径的，如果知道，还需要我们写代码干嘛，所以这就需要我们z对后面的路径长度做一个估计，即写一个**估价函数**来估计后面的路径，这就是启发式搜索的核心，一个估价函数的好坏决定了一个程序的效率。

估价函数

关键是估价函数怎么构造呢？有什么限制吗？既然后面的路径我都不知道距离，该如何估计呢？

我们先考虑 $n*n$ 的方格的深搜。已知起点为 s ，终点为 t ，此时我已经到达了点 p ，并且此前我得到的最近距离为 $\min$ 。那么我应该如何判断是否该继续走下去呢？

很容易想到 $L_{sp} \geq \min$ ，就返回，如果我们加上启发式搜索的思想呢？构造一个估价函数呢？

当前已经确定的路径长度为 L_{sp} ，而 L_{pt} 的长度是未知的，但是我们可以估计一下 G_{pt} ，只要 $L_{sp} + G_{pt} \geq \min$ ，我们就认定通过该点到达终点的距离不可能是最短距离，那么我们就可以提前结束搜索了。

那么首先我们先思考一下？估计的距离和真实的距离的关系？是一定要大于，还是一定要小于，还是随意指定。

估价函数

首先，想都不需要想，随意指定是肯定不可能的，比如我指定所有点到终点的估计距离都 k ，那么对搜索的效率而言是没有任何意义的，当然，一定相等也是不可能的，既然我都知道每个点到终点的真实距离了，还需要找最短路径干嘛。

我们也很容易想到，估计路径 $>$ 实际路径也是错误的，想一个极端的例子，我让所有的估计距离都为正无穷，那么肯定满足条件，但是呢，对搜索效率来说是没有任何帮助的。再想一下，如果 $G_{pt} > L_{pt}$ ，那么很有可能 $L_{sp} + L_{pt} < \min$ 但是 $L_{sp} + G_{pt} > \min$ ，那这样不但没有提高效率反而你的程序都是错误的了。所以估价函数必须满足的要求是：

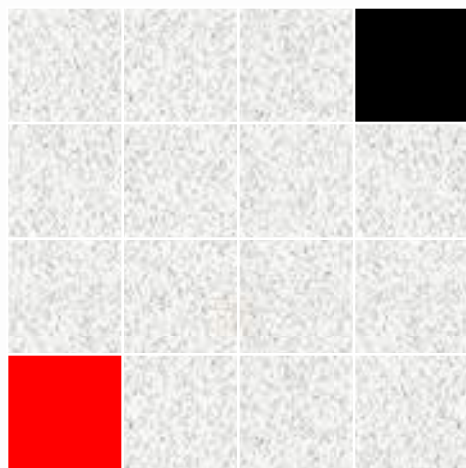
对于任意一个点：估计距离 $\leq$ 实际距离

并且估计距离越接近实际距离，效率越高

估价函数

我们思考一下，当估计距离=实际距离的时候，那么我们是不是可以不做任何无用功的直接一条路径搜索到从起点到终点的最短路径。

那我们再思考一下，对于前面说的 $n*n$ 的迷宫的深搜，我们如何构造估价函数呢？才能保证估计距离 $\leq$ 实际距离。



估价函数

是不是很容易想到，就是曼哈顿距离。

曼哈顿距离： $|x_1 - x_2| + |y_1 - y_2|$

也就是从一个点到另一个点，如果只能上下左右四个方向走，中间要经过的最短步数。

也就是说，如果当我到达点p的时候，如果后面没有任何障碍，后面每次都沿着最短路径走，到达终点的步数都 $\geq \min$ 的时候，那么是不是就可以说明这条路径肯定就不是最短路径了。这是一个很明显的道理。

当然，并不是说，每次搜索的时候，估价函数都是曼哈顿距离，这里只是举一个例子，只要你能保证估计距离 $\leq$ 实际距离，那么你的估价函数就是正确的，并且两者越接近，那么效率就越高。



搜索的优化

经过前面的学习，这也给我们做搜索题的时候提供了一种思路，我们不但要考虑当前最优状态，还需要对到达当前状态之后的剩余状态做一个估计，根据估价函数的设计原理，我们可以理解为，当到达当前状态时，如果后面每次我都通过理论上最优的办法去到达最终状态，如果效率比当前的最优解都还要差的话，那么这种状态就没有必要继续搜索下去了，这样就大大的节省了搜索的时间，提高了搜索的效率。



例题

给定一个1到n的排列 x ，每次你可以将 $x_1 \cdots x_i$ 翻转，问你需要多少次操作才能将该序列变为升序序列。 $N \leq 25$

例题

该题就是一道简单的搜索题，只要难度在于剪枝上面，我们要善于发现规律，对于每一次翻转，最多只能改变相邻数字发生改变的只有第一个数和第 i 个数，当第 i 个数与相邻的后一个数之差刚好为1时，这次交换导致求解的期望次数一定会+1.

交换之后，相邻的两个数之间的差如果 >1 ，那么肯定至少还需要一次交换才可以达到最终的效果，所以我们完全可以用启发式搜索，当前操作步数+后序估计步数(当前序列中 >1 的相邻数的对数) $>$ 当前最优答案

POJ 3460 Booksort

【例题】Booksort^{POJ3460}

给定 n ($1 \leq n \leq 15$) 本书，编号为 $1 \sim n$ 。在初始状态下，书是任意排列的。在每一次操作中，可以抽取其中连续的一段，再把这段插入到其他某个位置。我们的目标状态是把书按照 $1 \sim n$ 的顺序依次排列，求最少需要多少次操作。若操作次数 ≥ 5 ，则直接输出字符串“5 or more”即可。

解题思路

很容易看出这是一道搜索题，我们抽取长度为 i 的一堆书，取出来之后再放进去，放入的位置做最多有 $n-i+1$ 种，但是我们会发现每种方案都重复了一次。比如由1324，变为1234，即可以3后移，也可以2前移。所以中的方案数为 $i*(n-i+1)/2$ ($1 \leq i \leq n$)，当 $n \leq 15$ 时，每一次移动大概有560种可能。因为题目中要求操作次数不能大于等于5次，所以最多 560^4 。很明显会超时。

那我们该需要优化呢？第一种是双向bfs，从起点和终点同时向中间状态搜索2步，时间复杂度为 $2*560^2$ 。时间复杂度是可以接受的。

第二种是ID A*，也就是限制深度的迭代加深搜索。这种方法代码比较短，但是我们要思考如何才能估计后面的操作次数。

解题思路

和前面的A*一样，ID A*的估价函数也必须要满足前面所学习的估价函数的要求，也就是估计步数 $\geq$ 实际步数，否则估计就是错误的，并且越接近越好。

那么对于当前状态，我们要如何判断最少需要多少步才可能到达最终状态呢？

我们可以思考一下，一次操作最多可以改变多少本书的后继，假设这些操作都是正确的，也就是说每次改变刚好都把错误的后继变正确了。



如果我们移动一堆书，我们会发现改变了三个区域的后继：这堆书前面那本书的后继改变了；这堆书中最后一本书的后继改变了，插入的位置前的最后一本书的后继改变了。

解题思路

也就是说，当我们移动一堆书的时候，最多可以改变三本书的后继，也就是最多可以让三个错误的后继变为正确。那么启发式函数就好写了：

当前搜索层数+改变剩下的错误后继的最少次数>假设答案所在的层数。

解题思路

第一步：统计当前状态错误后继的个数。

```
if(a[1]!=1)
wa++;
for(int i=1;i<n;i++)
{
    if(a[i+1]!=a[i]+1)
    wa++;
}
```

第二步：确定当前状态最少需要的次数。

```
int num=(wa+2)/3;//表示最少次数，wa/3向上取整
```

解题思路

第三步：双重循环枚举取出哪一堆书。再枚举放入到哪个位置

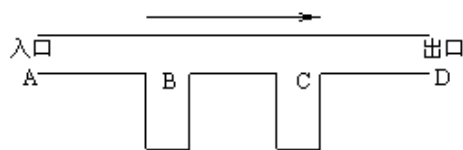
```
for(int i=1;i<=n;i++){  
    for(int j=i;j<=n;j++)//取出[i,j]之间的书  
    {  
        for(int k=1;k<=n+1;k++)//可以插入的位置有n+1个，去除区间[i,j+1]  
        {  
            if(k>=i&&k<=j+1)        continue;  
            if(k<i)  
                change(k,i-1,i,j);//相当于交换区间[l,i-1]和[i,j]的位置  
            else if(k>j+1)  
                change(i,j,j+1,k-1);//相当于交换区间[i,j]和[j+1,k-1]  
            递归回溯  
        }  
    }  
}
```

单向双轨道

题目描述

[展开](#)

如图所示 l ，某火车站有 B, C 两个调度站，左边入口 A 处有 n 辆火车等待进站(从左到右以 a, b, c, d 编号)，右边是出口 D ，规定在这一段，火车从 A 进入经过 B, C 只能从左向右单向开，并且 B, C 调度站不限定所能停放的车辆数。



从文件输入 n 及 n 个小写字母的一个排列，该排列表示火车在出口 D 处形成的从左到右的火车编号序列。输出一系列操作过程，每一行形如“ h, L, R ”的字母序列，其中 h 为火车编号， L 为 h 车原先所在位置(位置都以 A, B, C, D 表示)， R 为新位置。或者输出‘NO’表示不能完成这样的调度。

输入格式

一个数 $n(1 < n < 27)$ 及由 n 个小写字母组成的字符串。

输出格式

可以调度则输出最短的调度序列，当有多种方案时输出按操作(L, R)字典序最小的方案，不可以调度时则输出‘NO’。

解题思路

该题看上很难，实际上也很难，一道题不会做就想暴力怎么做。我们先思考一下每次挪动有哪几种可能，实际上就六种可能，A-B, A-C, A-D, B-C, B-D, C-D, 那么每次挪动肯定是这六种中的一种，而且既然要最小字典序最小，那么操作肯定是有顺序的。

我们既然要知道每次挪动的具体情况，那么我们只需要开四个栈就可以了，分别表示A, B, C, D, 因为只能从一头进出，那么就完全可以用栈模拟。

该题，我们考虑一下启发式搜索，既然每次都只可以挪动一个，那么D栈里面的盘子数量就可以用于启发式搜索。同样，该题可以用迭代加深搜索，这样第一次搜到的答案就是正解。以免操作错误，已知搜索下去浪费时间。

```
for(int i=1;i<=3*n;i++)//最多3*n层  
dfs(1,i); //ID-A*搜索
```

解题思路

```
if(s4[tp4]!=s[tp4])  
return ;//如果已经出栈的部分顺序和目标顺序不一样，那么直接返回  
if(step-1+(n-tp4)>num)//每一步最多出栈一个，如果每次都出栈还搜不到终点
```

```
if(tp1)//A--B  
{  
    l[step]='A';  
    r[step]='B';  
    h[step]=s1[tp1--];  
    s2[++tp2]=h[step];  
    dfs(step+1,num);  
    s1[++tp1]=h[step];  
    tp2--;  
}
```

```
if(tp3)//C--D  
{  
    l[step]='C';  
    r[step]='D';  
    h[step]=s3[tp3--];  
    s4[++tp4]=h[step];  
    dfs(step+1,num);  
    s3[++tp3]=h[step];  
    tp4--;  
}
```


深搜求方案数

深搜既然是尝试所有方案的可能性，那么只要能搜到终点的都是最短路，我们完全可以先让其搜到终点之后在判断和当前最短路的大小关系，如果相等，那么方案数+1，如果小了，那么说明前面记录的方案都是错误的，方案数=1.

同样，我们也可以在途中就进行剪枝，但是注意剪枝的时候如果两条路径相同需要继续搜索下去，不能返回。

深搜求方案数

```
void dfs(int x,int y,int step)
```

```
{
```

```
    if(step>Min)//剪枝
```

```
    return ;
```

```
    if(到达终点)
```

```
    {
```

if(step==Min)//假设当前最小值就是最优值，存在一条和他一样优的结果，方案+1
 方案数+1;

else//前面的最小值不是最优值，说明前面的方案都不符合条件，只有当前方案最优
 方案数=1;

```
    }
```

```
}
```

深搜求方案数

```
void dfs(int x, int y, int step)
{
    if (step <= map[x][y]) //相等不能返回
        map[x][y] = step;
    else
        return ;
    if (到达终点)
    {
        if (step == map[x][y])
            方案数+1;
        else
            方案数=1;
    }
}
```

深搜求具体方案

如果一条路径是从起点到最终点的最短路径，那么这条路径上的任意一个点通过这条路径到起点都一定是最短路径。所以如果我们要知道最短路的具体方案，我们可以通过记录该点由那个点到达或者该点可以到达哪个点来实现。

如果记录该点可以到达哪个点。那么第一个问题是从每一个点出发都可以到达不止一个点，那么就需要记录不止一个点，并且这些点也不一定能到达终点，所以比较麻烦

如果记录该点由哪个点到达，因为我们搜索的时候走的都是当前走过的路径的最短路，所以如果有路径更新，那么就证明最短路更新了，那么到该点的最短路的前驱也就变了，否则不发生改变。而且，到达终点的点一定是从起点出发的，也就是我们记录前驱，从终点开始搜索一定可以到达起点。

当然，路径不止一条，所以一般会限制搜索方式。

深搜求具体方案

```
if(step>=map[x][y].step)
{// 前驱不变
    return ;
}
else
{// 前驱改变, 更新
    map[x][y].step=step;
    map[x][y].firstx=firstx;
    map[x][y].firsty=firsty;
    if(x==n&&y==m)
    {
        return ;
    }
}
```

```
int k=1;
while(x!=0&&y!=0)
{
    a[k]=x;
    b[k]=y;
    tx=x;
    ty=y;
    x=map[tx][ty].firstx;
    y=map[tx][ty].firsty;
    k++;
}
for(int i=k-1;i>=1;i--)
    printf("%d %d\n",a[i],b[i]);
```

栅栏

题目描述

农夫约翰打算建立一个栅栏将他的牧场给围起来，因此他需要一些特定规格的木材。于是农夫约翰到木材店购买木材。可是木材店老板说他这里只剩下少部分大规格的木板了。不过约翰可以购买这些木板，然后切割成他所需要的规格。而且约翰有一把神奇的锯子，用它来锯木板，不会产生任何损失，也就是说长度为10的木板可以切成长度为8和2的两个木板。

你的任务：给你约翰所需要的木板的规格，还有木材店老板能够给出的木材的规格，求约翰最多能够得到多少他所需要的木板。

输入

第一行为整数 m ($m \leq 50$) 表示木材店老板可以提供多少块木材给约翰

后面 m 行为老板提供的每一块木板的长度。

接下来一行为整数 n ($n \leq 1000$)，表示约翰需要多少木材。

后面 n 行表示他所需要的每一块木板的长度。木材的规格小于32767。（对于店老板提供的和约翰需要的每块木板，你只能使用一次）。



栅栏

答案是具有单调性的，所以该题直接二分答案，然后再用dfs搜索剪枝去验证。



必做练习题

先去做两道深搜求方案数和具体路径的水题

MSOJ	1213	靶形数独
MSOJ	1663	小猫爬山
MSOJ	1667	分割矩阵
洛谷	4438	道路
MSOJ	1423	送礼物
洛谷	1139	单向双轨道
洛谷	2329	栅栏